How To Explain This Project

What to say, at four different lengths, plus the questions this project attracts and short answers to them.

The one fact everything hangs on

> Fraud is **0.17%** of the data. A model that answers "legitimate" for every single transaction scores **99.83% accuracy** and catches **zero** fraud.

If you only get one idea across, make it this one. It explains why accuracy is never reported, why recall and precision are used instead, and why the decision threshold is chosen from cost rather than left at 0.5. Lead with it.

In 30 seconds

> We built a system that detects fraudulent credit card transactions. The hard part isn't the classification — it's that fraud is only 0.17% of the data, so a model can be 99.8% accurate and catch nothing. We trained models from every unit of the syllabus on one identical split and compared them on recall and precision instead of accuracy. XGBoost was best: it catches 116 of 142 frauds while raising only 6 false alarms.

In 2 minutes

Add, in this order:

1. **The data.** 284,807 real transactions from European cardholders, 492 fraudulent. We removed 1,081 duplicate rows first, leaving 283,726 with 473 frauds. V1–V28 are PCA components — the bank anonymised the original features for privacy.
2. **The protocol.** One stratified 70/30 split, random seed 77, reused by every model. Scaling and resampling happen inside the pipeline so nothing from the test set leaks into training. This is what makes the comparison fair.
3. **The metrics.** Recall, precision, F1 and PR-AUC. Not accuracy, for the reason above.
4. **The result.** Ensembles beat single learners. XGBoost: recall 0.817, precision 0.951, F1 0.879. Five-fold cross validation confirmed the ranking, so it isn't a fluke of one split.
5. **The demo.** Point at the screen and run it.

In 10 minutes

Walk the six core notebooks in order — 01, 03, 04, 05, 08, 10 — spending about a minute on each, then finish with the live demo. The table below gives you the one sentence that matters for each.

What each notebook is for

#	Notebook	The one thing to say about it
01	Data Exploration and Preprocessing	“This is where we found the imbalance and proved accuracy was useless.”
03	Decision Tree (ID3)	“Entropy and information gain, with the depth chosen by cross validation. Depth 6, 29 leaves. Its root split is V17 ≤ −2.53.”
04	K-Nearest Neighbour	“No training phase at all — it stores the data and compares distances. We also used it to demonstrate the curse of dimensionality on real data.”
05	Naive Bayes and Bayes Optimal Decisions	“Bayes theorem with the naive independence assumption. This is also where we derive the cost-based threshold.”
06	Support Vector Machine	“Margin-based classification, linear and RBF kernels compared.”
07	Artificial Neural Networks	“McCulloch–Pitts up to backpropagation. We show a single perceptron failing on XOR, exactly as the theory predicts, and the MLP solving it.”

#	Notebook	The one thing to say about it
08	Ensemble Learning	“Bagging and boosting. Random Forest, AdaBoost and XGBoost. This is where the best result comes from.”
10	Model Comparison and Results	“Everything on one test set, plus the cross-validated check.”
02	*(Extras)* Feature Selection and Reduction	“Supporting analysis — which features matter, PCA and LDA.”
09	*(Extras)* Clustering	“Can fraud be found without labels at all? Partly — K-means separates some of it.”

The syllabus, unit by unit

Useful if sir asks “where is Unit III in this?”

Unit	Where it is
I — training, testing, validation, overfitting	Notebook 01, and `Code/utils.py` which holds the shared split
II — decision trees, k-NN, feature selection, PCA/LDA	Notebooks 03, 04, and 02 in Extras
III — Bayes theorem, Bayes optimal decisions, SVM	Notebooks 05 and 06
IV — perceptron, backpropagation, learning theory	Notebook 07
V — bagging, boosting, rule fusion, clustering	Notebooks 08 and 09

Numbers to know without looking

Transactions	284,807 raw → **283,726** after removing 1,081 duplicates
Frauds	492 raw → **473** after dedupe (**0.1667%**)
Split	70/30 stratified, seed **77** (last two digits of AP24110010677)
Test set	85,118 transactions, **142** frauds
Trivial classifier	**99.83%** accuracy, 0 frauds caught
Best model	XGBoost — recall **0.817**, precision **0.951**, F1 **0.879**
What that means	**116 of 142** frauds caught, **6** false alarms

---

Running the demo in front of someone

Do it in this order. It takes about 90 seconds.

1. Open on Tab 3 — "Why we do not report accuracy." Read the 99.83% out loud. Let it sit for a beat. This frames everything else.

2. Tab 1, switch the model to Decision Tree, click "Show a real fraud." The panel on the right prints the exact chain of questions the tree asked: *Is V17 ≤ −2.53? yes. Is V12 ≤ −2.18? yes.* Say: "this is the one model whose reasoning you can read directly — those are the same splits printed in notebook 03." This is the strongest moment in the demo. Don't rush it.

3. Switch to XGBoost on the same transaction. Different model, different probability, same verdict. Shows the comparison is live, not a screenshot.

4. Tab 2, click "Run the model", then drag the threshold slider down. Frauds caught goes up, false alarms go up with it. That is the cost trade-off made physical. Say: "this is why the threshold is a decision about cost, not a default."

If something goes wrong on the day, Tab 3's results table is read from the saved results files, not from the models, so it works even if a model fails to load. You can present the whole comparison from that one tab.

Questions this project attracts

Why not report accuracy? At 0.17% fraud, answering "legitimate" every time already scores 99.83%. Accuracy measures the wrong thing when one class is this rare.

Why PR-AUC and not ROC-AUC? ROC-AUC uses the false positive rate, whose denominator is the enormous legitimate class. That denominator swamps everything, so

ROC-AUC looks good even for a weak model. PR-AUC uses precision, which reacts to false positives directly.

What is data leakage, and where could it happen here? Leakage is test information reaching the model during training. Three places it could happen here: scaling before splitting, oversampling before splitting, and duplicate rows landing in both halves. We deduplicate first, split second, and keep scaling and resampling inside the pipeline so they only ever see the training fold.

What is the Bayes optimal decision threshold? *(Unit III, notebook 05)* If missing a fraud costs far more than a false alarm, you shouldn't wait until you're 50% sure before acting. With a false negative costing 50 times a false positive, the break-even point is 1/51, about 0.02. So the model flags anything above 2% probability instead of 50%.

Why did the probabilities need calibrating first? Because that calculation assumes the model outputs real probabilities. Naive Bayes pushes its outputs very close to 0 and 1, so its "90%" isn't really 90%. Isotonic calibration rescales them to match observed frequencies, after which the threshold behaves as the theory predicts.

What's the difference between bagging and boosting? *(Unit V)* Bagging trains many models independently on random subsets and averages them — it reduces variance. Boosting trains models one after another, each correcting the previous one's mistakes — it reduces bias. Random Forest is bagging; AdaBoost and XGBoost are boosting.

What is stacking? *(Unit V)* Instead of combining predictions with a fixed rule like majority vote, a second model is trained to learn the best way to combine them.

What's the difference between PCA and LDA? *(Extras, notebook 02)* PCA finds the directions with most spread and ignores the labels. LDA finds the directions that best separate the two classes, so it uses them. PCA is unsupervised, LDA is supervised.

What is VC dimension? *(Unit IV, notebook 07)* A measure of model complexity — the largest number of points the model can separate in every possible arrangement. It's used to estimate how much training data a model of that complexity needs before its test performance can be trusted.

Why seed 77? Last two digits of roll number AP24110010677, as instructed. It fixes the split and every random choice, so anyone running the repository gets the numbers in the report.

Why is V1 to V28 meaningless? Can't you interpret the features? No, and that's a real limitation we state. The bank could not release the raw features for privacy reasons, so the publishers applied PCA and released the components. Only Time and Amount are untransformed.

Could this be deployed? Not as it stands. It's trained on two days of 2013 data from one region, fraud patterns drift, and a real system would need retraining on a schedule plus

a human review queue. The modelling is sound; the productionisation is out of scope.

Things to own rather than hide

Being straight about these is stronger than hoping nobody asks.

The demo's XGBoost may differ slightly from the report. > "The demo retrains on this laptop, which has a newer XGBoost than the machine > that produced the report. It costs one extra false alarm — 7 instead of 6. > Recall is identical."

Tab 2's precision looks better than reality. The demo sample is deliberately fraud-heavy — 142 frauds in 1,642 transactions — so there's something to look at. The app says so on screen. The true figures are in the sidebar and Tab 3, measured on the full 85,118-transaction test set.

Random Forest has lower recall than the plain Decision Tree. 0.725 against 0.782. It is not a mistake. Random Forest is far more precise (0.972 against 0.867, 3 false alarms against 17), so it is more conservative about calling something fraud. Which you prefer depends on what a false alarm costs you — which is exactly the threshold argument.

Accuracy is not in the results table. Deliberate, not an oversight. That's the whole point of the project.